Statement of Contribution

COMP 3019J – Project

Week Number: 9

Milestone Number: 1 (Design Document)
Group Number: 10

We declare that the following contribution to the project for the Web Application Development module is correct, and agreed to by the entire team. We accept that this breakdown of contribution will affect the distribution of grade for the project.

We further accept that in case these contributions are found to be falsified / misleading / incorrect, by any means, now or in the future, the team as a whole is liable. We understand that penalty for such action could range from failing the project to a referral to the disciplinary committee.

Project Name: The Green Kitchen

Contribution (%) of Teammate #1: 33%

Contribution (%) of Teammate #2: 33%

Contribution (%) of Teammate #3: 33%

(must add up to 100%)

	Teammate #1	**Teammate #2**	**Teammate #3**
Name (in capital)	CIARA BEHAN	EVA BARRETT	ZHAO WEIYE
UCD-ID	23441364	23733661	23219648
UCDconnect Email	ciara.behan@ucdconnect.ie	eva.barrett@ucdconnect.ie	weiye.zhao@@ucdconnect.ie

Teammate #1 Signature: *Ciara Behan*

Teammate #2 Signature: *Eva Barrett*

Teammate #3 Signature: *Zhao Weiye*

Date: 05/11/2025

The Green Kitchen

Design Document

Introduction

The way we produce food is rapidly degrading our planet, contributing to 70% of biodiversity loss on land, 50% in freshwater ecosystems, and around 30% of global greenhouse gas emissions. Some foods have a far greater environmental impact than others, and if everyone made even small adjustments to their diets, the world could move toward a far more sustainable future.

“The Green Kitchen” is a platform designed to expose users to sustainable recipes and encourage the substitution of common household ingredients with more eco-friendly, equally tasty alternatives. Creators can share their recipes, which are then assigned a Sustainability Score. With a simple click, creators can access suggestions for more eco-friendly ingredient swaps.

Users can also engage with each other through comments, offering further ideas and feedback on each recipe. An admin team regularly updates the database of ingredient swaps, ensuring the platform continuously evolves to promote even greater sustainability.

Website goals:

- Hosts sustainable-forward recipes contributed by the community.
- Gives every recipe a Sustainability Score based on its ingredient footprint.
- Suggests ingredient swaps to reduce impact (e.g., replace beef mince with lentils + mushrooms).

- Enables discussion via comments and reactions.

- Restricts/extends features by user type (Readers, Creators, Admins).

Functionality

User Authentication & Account Management

Users can register, log in, log out, and interact with recipes according to their assigned role:

- Reader – can view all recipes, comment, and bookmark favourites.
- Creator – has all Reader abilities, plus can create and edit their own recipes.
- Admin – manages users, recipes, and comments, with full moderation privileges.

Unregistered users (guests) can freely browse the site and view recipe content, but are prompted to sign up when attempting to post a comment or add a recipe. During registration, users choose whether to sign up as a Reader or Creator using a checkbox on the sign-up form. This selection determines which features they can access once logged in. Later, users can visit their Profile page to update their information and switch between Reader and Creator roles if they decide to start posting their own recipes.

<u>Implementation Details:</u>

- Users register with email and password, which are hashed using Werkzeug's bcrypt utilities before storage.
- The User table includes a role column (reader, creator, admin) enforced by a SQLAlchemy CheckConstraint.
- Session management is handled using Flask-Login
- Routes requiring specific permissions are protected by decorators such as @login_required and @require_role
- Guest users can view recipes but cannot post comments or create recipes
- If we have time we would also like to include email verification for new accounts, "soft deletion" and account recovery, and profile customisation.

Files Needed:

- models.py – defines the User model and role validation.
- blueprints/api/auth.py – handles registration, login, logout, and optional role updates.
- templates/login.html, templates/register.html, templates/profile.html – user-facing authentication pages.
- static/js/utils.js – form validation and AJAX helper functions.

Recipe Creation & Editing

Creators and admins can post and edit recipes containing ingredients, quantities, and preparation steps. Each recipe receives a recipe_score automatically, reflecting the sustainability of its ingredients.

Technical Implementation:

- Form data submitted via POST /api/recipes.
- Backend validates fields, verifies role permissions, and stores data using SQLAlchemy ORM.
- A Scoring Service calculates the average ingredient_score (recipe_score = average(ingredient_score)) for all linked ingredients and stores this in the database as the recipe_score.
- Optional: step-by-step instructions, cooking time, difficulty, and sustainability tags (e.g. *zero-waste*, *local ingredients*).

Files Needed:

- models.py – Recipe and RecipeIngredient models.
- blueprints/api/recipes.py – Create, edit, and retrieve routes/endpoints.
- services/scoring.py – Computes and updates recipe_score using compute_recipe_score() function
- templates/recipe_form.html, static/js/recipe_form.js.

Viewing Recipes.

Technical Implementation:

- GET /recipes renders via Jinja templates with server-side pagination.
- Recipes are displayed with color-coded sustainability badges (1-3: green, 4-6: yellow, 7-10: red).
- AJAX requests load additional data (comments, swaps, bookmarks) dynamically.
- Unregistered users can browse recipes but are prompted to sign up to see detailed sustainability data.

Files Needed:

- templates/recipes_list.html, templates/recipe_detail.html
- blueprints/web/routes.py – HTML/Frontend routes
- blueprints/api/recipes.py – JSON data for swaps/comments
- static/js/recipe_detail.js – dynamic fetch and render logic

Ingredient Scoring

Technical Implementation:

- Scores are stored in the ingredients table.
- Recipes use these values when computing recipe_score.
- Seed scripts prepopulate a base list of ingredients and their scores.

Files Needed:

- models.py – Ingredient model
- seeds/seed_ingredients.py – populate ingredient data
- services/scoring.py – fetch and average ingredient scores

Ingredient Swaps

Technical Implementation:

- The SWAPS table links a from_ingredient_id to a to_ingredient_id with a rationale.
- AJAX requests (GET /api/swaps?from=id) fetch swap suggestions, displayed
- under each ingredient in the recipe view.

Files Needed:

- models.py – Swap model
- services/swaps.py – lookup helper
- blueprints/api/swaps.py – API endpoint
- static/js/recipe_detail.js – AJAX fetch + render swaps

Comments (AJAX)

Technical Implementation:

- POST /api/recipes/:id/comments accepts JSON{body}
- Flask validates the logged-in user and inserts a Comment record via SQLAlchemy.
- The API returns the new comment in JSON; JS appends it to the page DOM.

Files Needed:

- models.py – Comment model
- blueprints/api/comments.py
- templates/recipe_detail.html – comment section
- static/js/comments.js – AJAX form handling

Bookmarks

Technical Implementation:

- AJAX call to POST /api/bookmarks/:recipe_id/toggle.
- Flask checks login, adds or deletes a Bookmark entry, and returns JSON {bookmarked: true|false}.

Files Needed:

- models.py – Bookmark model
- blueprints/api/bookmarks.py – API routes
- templates/bookmarks.html – display list
- static/js/bookmarks.js – toggle button logic

Admin Moderation

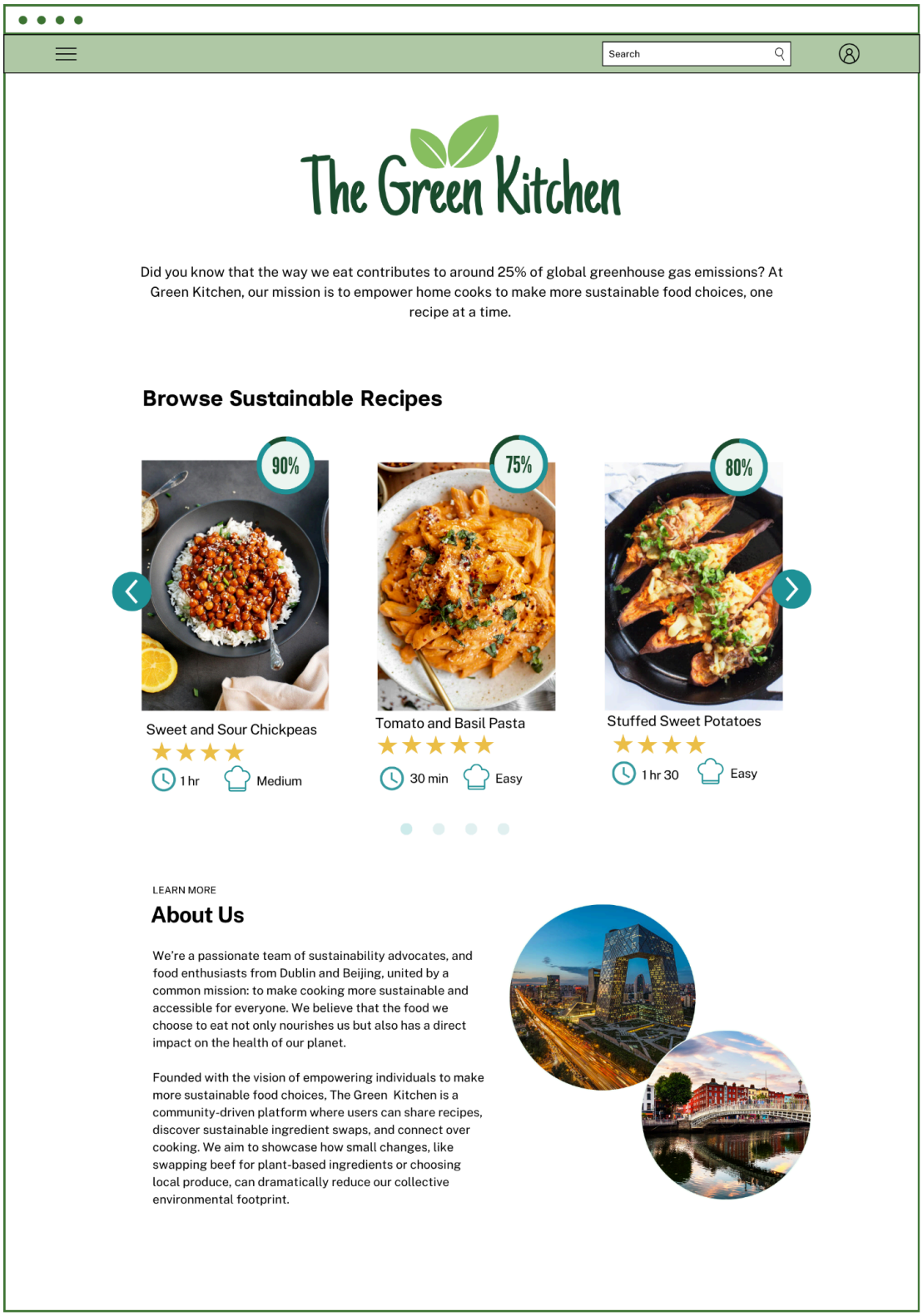
Admins can manage users, recipes, and comments (edit, hide, or delete).

- Technical Implementation:
- Admin-only routes (@require_role('admin')).
- Simple dashboard listing all content with action buttons triggering backend endpoints.

Files Needed:

- blueprints/admin/routes.py
- templates/admin_dashboard.html

Web Page Mockups



Recipe Page (Reader View)

Search

Stuffed Sweet Potatoes

Author: User Name

+ Save Recipe

244 User Ratings

Serves: **4 - 6**

Difficulty: **Easy**

Prep: **20 mins**

Cook: **1 hr 10 mins**

Sustainability Rating: **80%**

Ingredients

- 1 tbsp olive oil
- 2 onions
- 3 sweet potatoes
- 100g cheddar cheese

Method

1. Peel potatoes, then boil for 40 minutes
2. Finely dice onions, then sauté until golden
3. Drizzle potatoes with olive oil, add the onions, then grate the cheese on top
4. Bake on high heat for 50 minutes, flipping every 15 minutes
5. Plate
6. Serve and enjoy!

Comments (4)

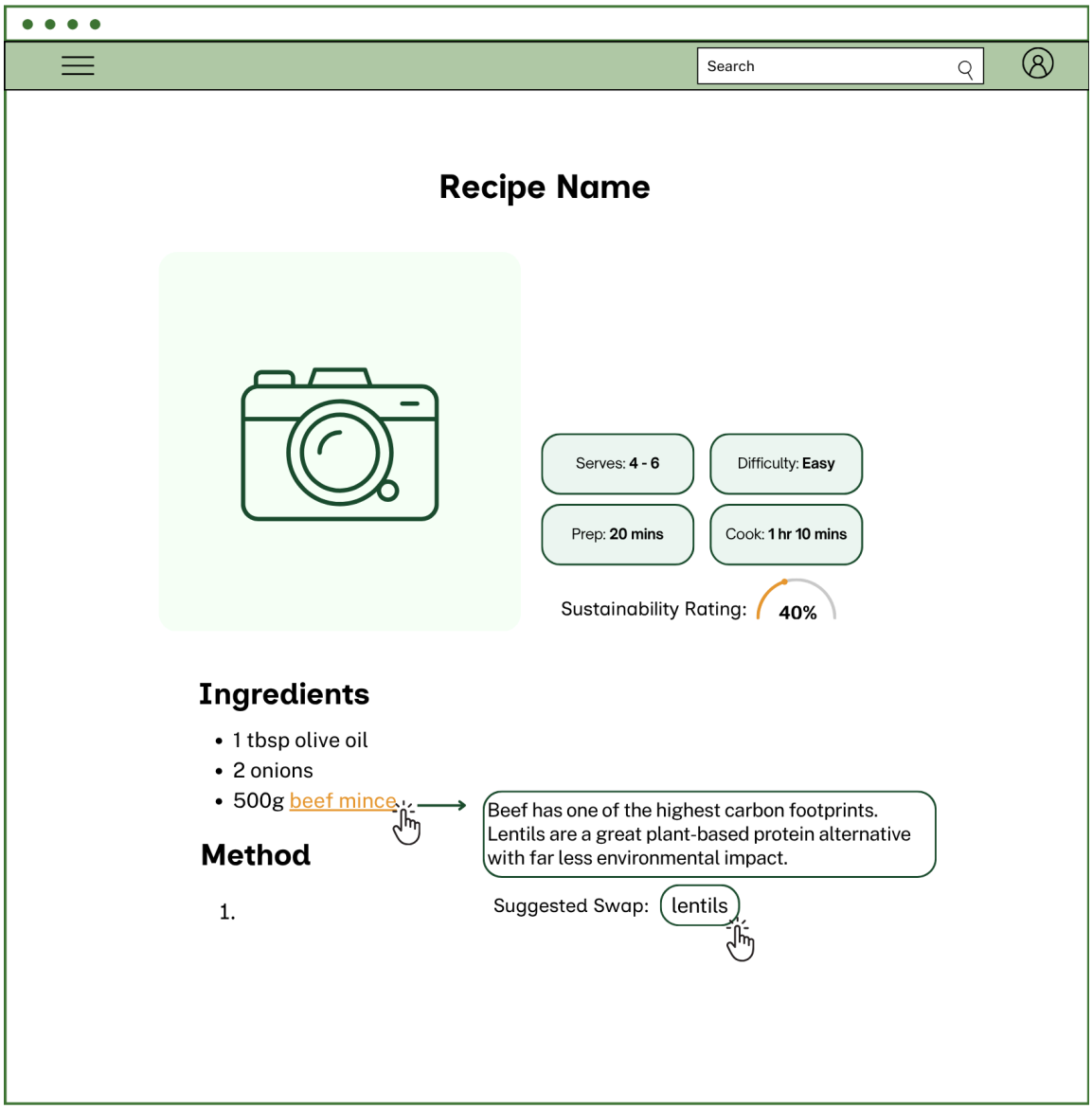
Easy and delicious!

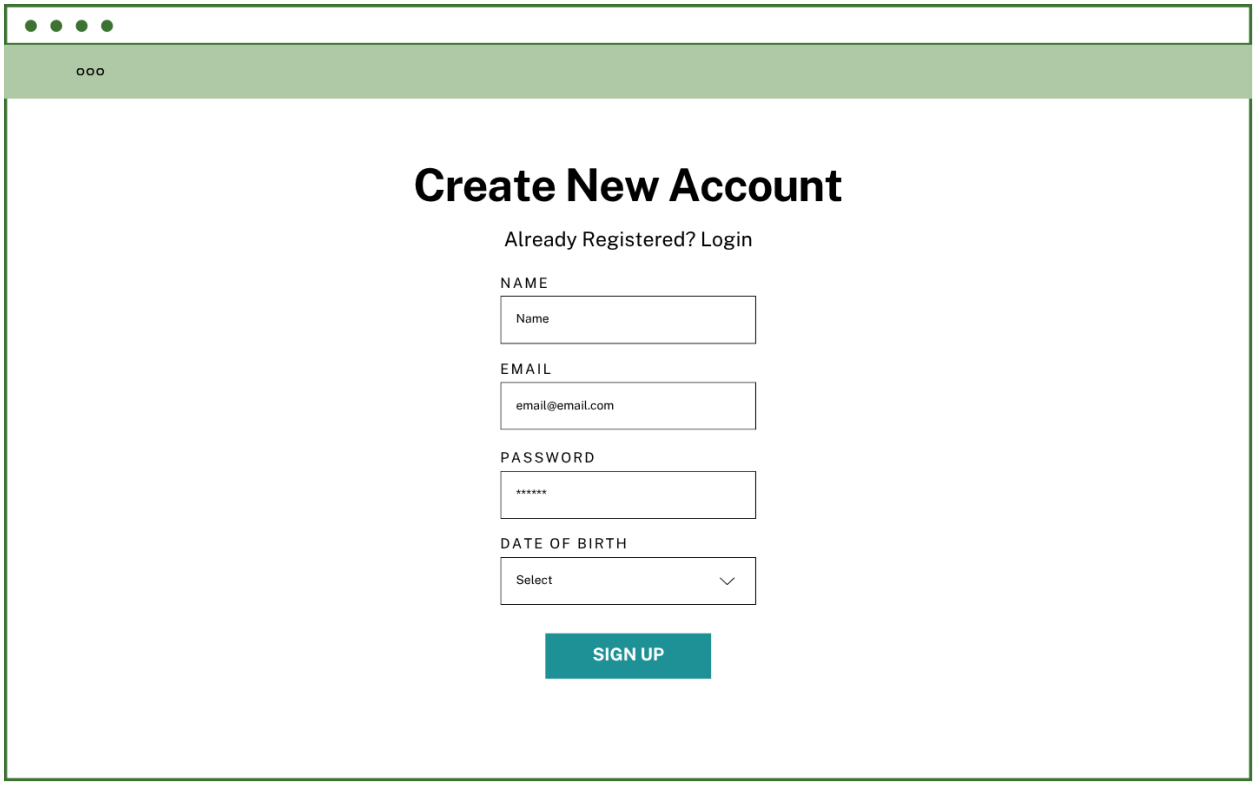
Easy and delicious!

Easy and delicious!

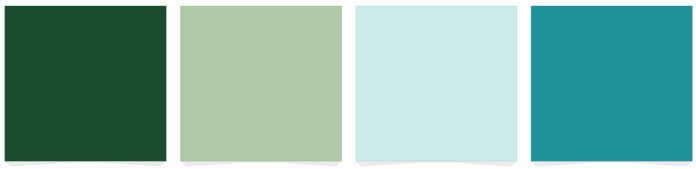
Easy and delicious!

Recipe Page (Writer View)





Color Palette



#1c4c2d #afcca9 #cfedee #229599

Database Diagrams

Architectural Pattern of Project

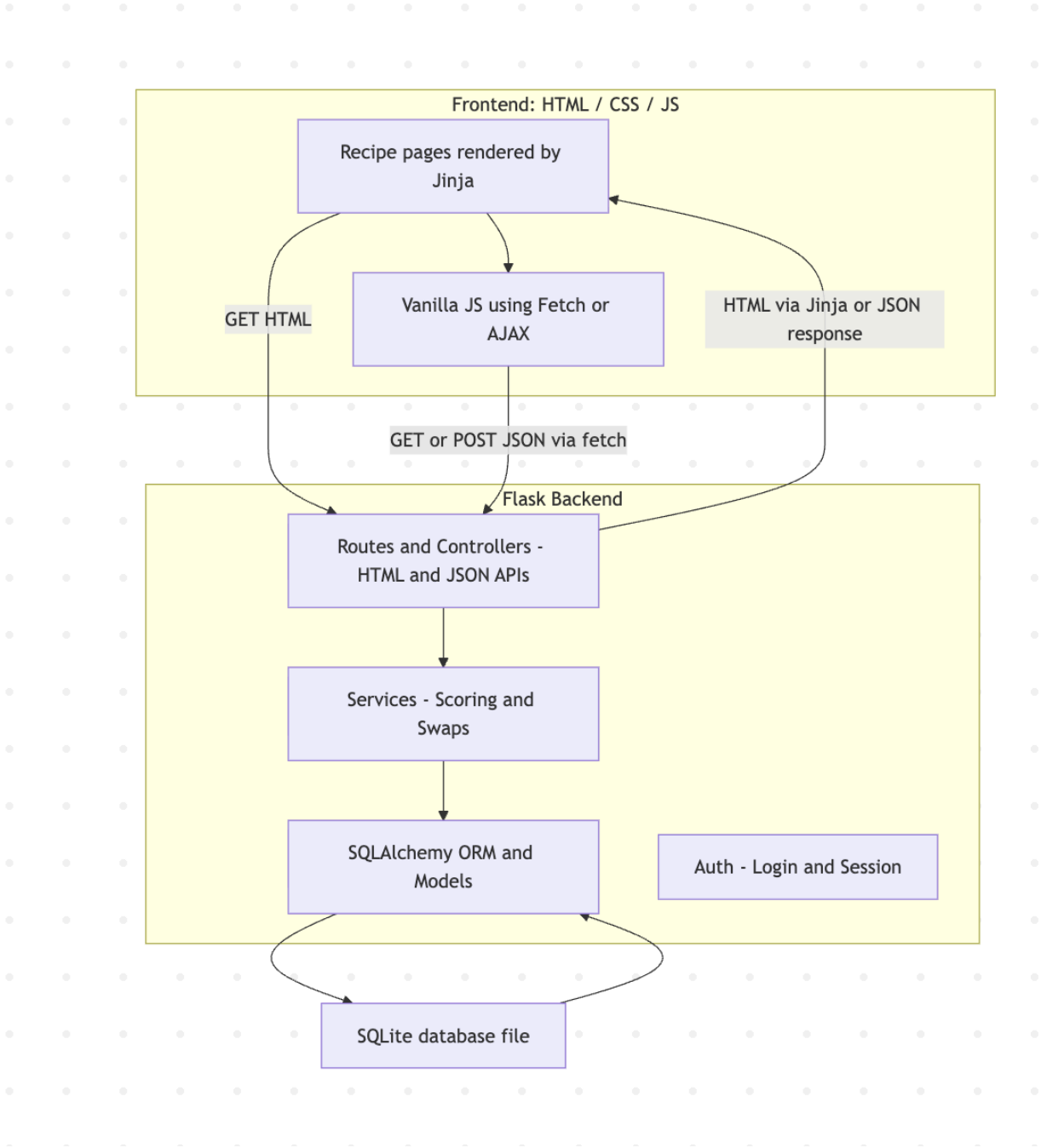


***Figure 1**: client-server architecture*

As evidenced by *Figure 1,* users interact through the frontend built with HTML, CSS, and JavaScript, which communicates with the Flask backend using both standard HTTP requests for full HTML pages (rendered via Jinja templates) and AJAX/Fetch JSON calls for dynamic features such as posting comments or fetching ingredient swap suggestions.

The backend follows a modular design, separating Routes/Controllers, Service Logic, and Data Models managed through SQLAlchemy ORM.

All data, including users, recipes, ingredients, swaps, and comments, is persisted in a local SQLite database during development, as recommended for Flask coursework.

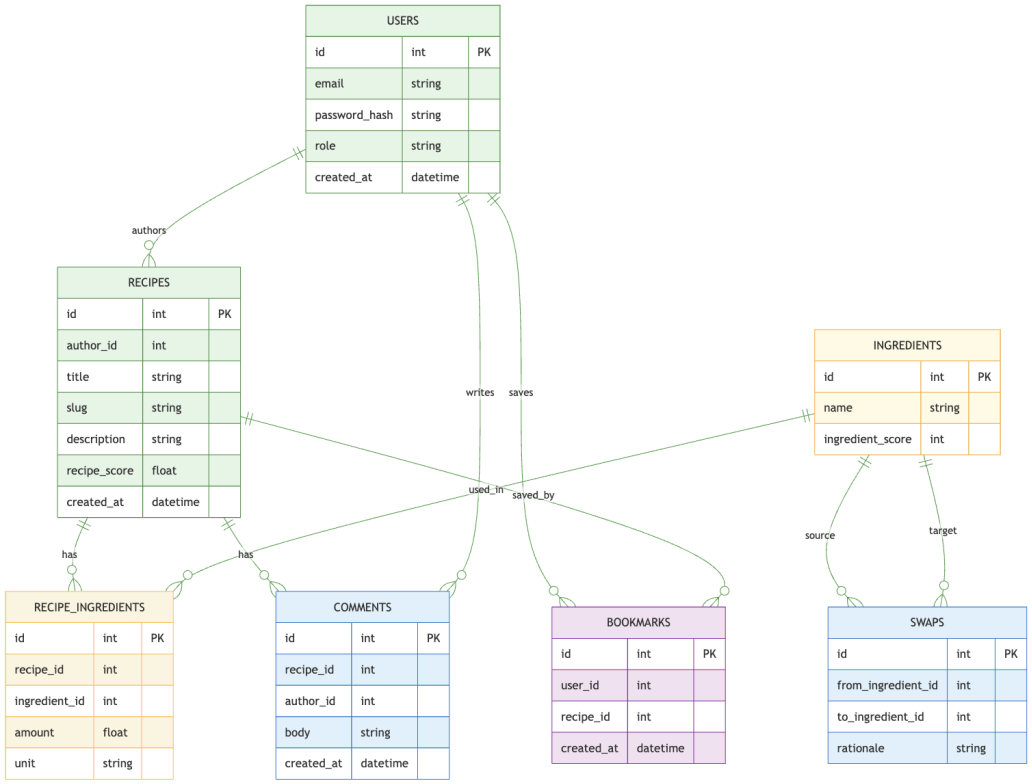


***Figure 2**: ER Diagram*

Figure 2 illustrates the relational database structure for our Website. It shows how users, recipes, ingredients, and related entities interact within the system. The core entities are Users, Recipes, and Ingredients.

Users Table

Each User can create multiple Recipes and post Comments on any recipe. In the database, the users table stores authentication information along with a role value, represented as text. We implement three distinct roles (Reader, Creator, and Admin) and enforce their permissions within the Flask application layer. Application-level route guards ensure these permissions are respected. For example, the POST /recipes endpoint requires a Creator or Admin role.

Recipe & Recipe_Ingredients Tables

Each Recipe is composed of multiple Ingredients, managed through the Recipe_Ingredients linking table, which stores ingredient quantities and measurement units.

Ingredients Table

Each Ingredient includes an ingredient_score field (on a 1–10 scale), while each Recipe stores a cached recipe_score which is calculated as the average (or weighted average) of its ingredients when created or updated, and this value is cached for fast access.

Swaps Table

The Swaps table suggests more sustainable alternatives for existing ingredients.

Bookmarks Table

The Bookmarks table represents a many-to-many relationship between users and recipes, allowing users to save or revisit recipes they like.

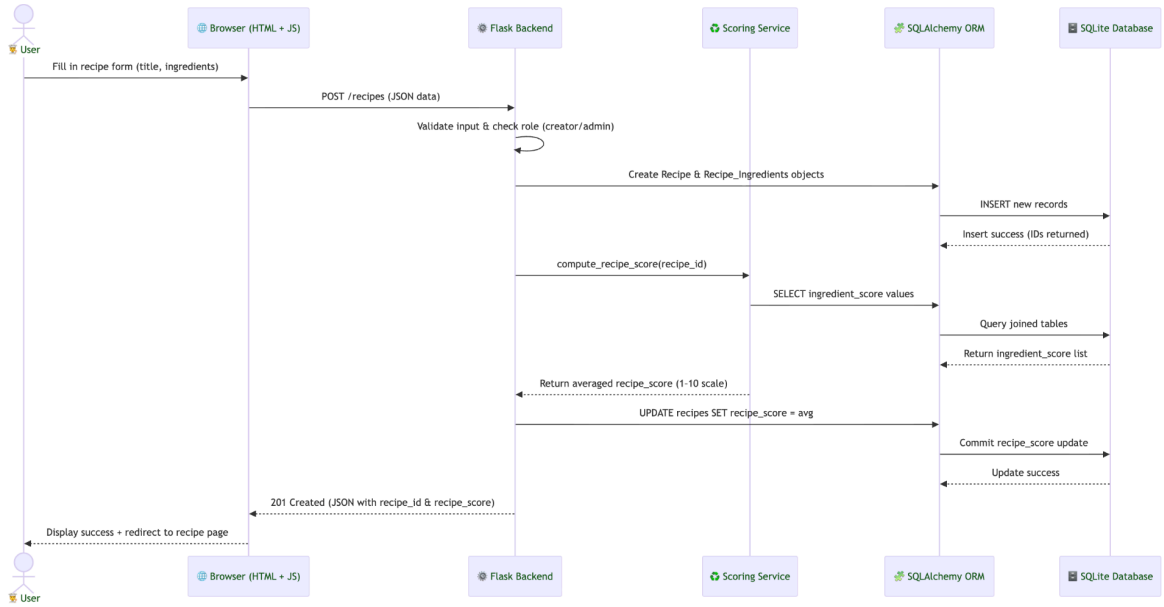


***Figure 3**: sequence diagram for posting a new recipe*

Figure 3 illustrates the end-to-end sequence of interactions that occur when a user submits a new recipe through the website. It demonstrates how the frontend, backend, and database layers communicate to create and score a recipe based on its ingredients.

The process begins when the User fills in the recipe form and submits it through the Browser. The browser sends a POST request to the Flask Backend, which first validates the input and checks the user's role (only users with the Creator or Admin role are permitted to post recipes).

Flask then uses the SQLAlchemy ORM to create new Recipe and Recipe_Ingredients records in the SQLite database. Once these are saved, the backend's Scoring Service retrieves each ingredient's ingredient_score and calculates the overall recipe_score. This value is cached within the recipe record for efficient future access.

Finally, Flask updates the database with the computed recipe_score and returns a success response to the browser. The Browser then displays a confirmation message or redirects the user to their recipe page.

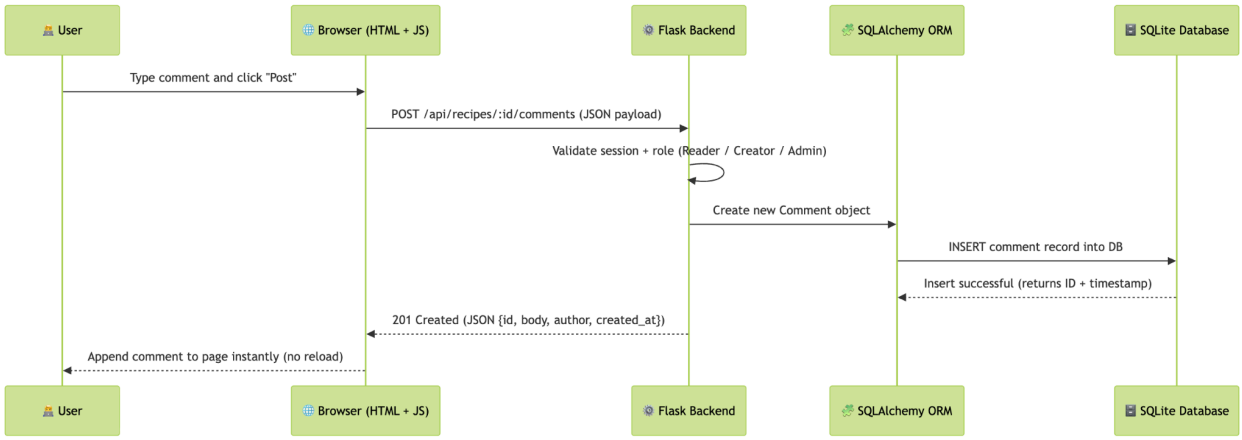


***Figure 4**: sequence diagram for adding a comment*

Figure 4 illustrates the asynchronous process that occurs when a logged-in user posts a comment on a recipe. When the user clicks post, the Browser then sends an AJAX request (using the Fetch API) to the Flask Backend, containing the comment data in JSON format.

Upon receiving the request, Flask validates the user's session and role (only authenticated users can post comments). If validation passes, Flask uses SQLAlchemy ORM to create a new Comment record and insert it into the SQLite database. Once the database operation is successful, Flask returns a JSON response containing the comment details (ID, body, author, and timestamp).

The Browser then dynamically updates the recipe page by appending the new comment to the existing list without reloading the entire page.